

HIGH-PERFORMANCE & PARALLEL COMPUTING

*Laboratory Practice Manual: Shared-Memory Concurrency & OpenMP Paradigms
(Comprehensive Python & Java Student Editions)*

HONOR CODE & LAB SUBMISSION POLICY

Students are encouraged to discuss high-level theoretical concepts and architectural bottlenecks with peers; however, all submitted implementations, profiling benchmarks, empirical graphs, and analytical write-ups must represent original individual work. Automated code comparison tools and benchmark replication checks will be enforced.

1. Conceptual Overview: OpenMP Paradigms in Python and Java

OpenMP (Open Multi-Processing) has served as the definitive API for shared-memory parallel programming in C, C++, and Fortran since 1997. It provides a directive-based fork-join concurrency model where a master thread spawns a team of worker threads to execute structured blocks of code cooperatively. While neither Python nor Java provides native compiler support for '#pragma omp' directives, the fundamental concurrency patterns pioneered by OpenMP—such as fork-join team creation, loop work-sharing, reductions, scheduling policies, cache-line false sharing, and recursive task stealing—map directly into modern high-performance runtimes for both Python and Java.

The objective of this laboratory manual is to bridge the gap between traditional OpenMP compiler directives and their idiomatic implementations in Python (via Cython's openmp wrapper and Numba's OpenMP-backed parallel compiler engine) and Java (via the Fork/Join Framework, Parallel Streams, and JVM cache controls).

1.1. Cross-Language OpenMP Mapping Architecture

The following reference table outlines how OpenMP directives correspond to constructs and architectural mechanisms across languages:

OpenMP Directive (C/ C++)	Python Equivalent (Numba / Cython)	Java Equivalent (JVM / Concurrency)	Architectu ral Mechanis m
#pragma omp parallel	@njit(parallel=True) / nogil parallel()	ForkJoinPool.submit() / Thread[]	Thread team creation, OS thread pooling
#pragma omp parallel for	numba.prange(N) / cython.parallel.prange	IntStream.range().parallel()	Loop range partitioning & thread distribution
reduction(+:var)	Inferred total += in prange	DoubleStream.sum() / reduce()	Private accumulators merged via tree reduction

#pragma omp critical	threading.Lock() / cython.critical	synchronized (lock) / ReentrantLock	Mutual exclusion lock & serialization
#pragma omp atomic	cython.parallel.atomic / Atomic primitives	java.util.concurrent.atomic. *	Hardware atomic bus lock (e.g. CMPXCHG)
schedule(static, chunk)	Uniform slicing / cython static	Static index partitioning (N / P)	Deterministic loop block assignment
schedule(dynamic, chunk)	Dynamic task queue / work dispenser	AtomicInteger fetch- and-add queue	Self-scheduling work queue with contention
#pragma omp task / taskwait	concurrent.futures / custom queue	ForkJoinTask / RecursiveTask.invokeAll	Dynamic work-stealing task deque
#pragma omp barrier	threading.Barrier / cython.barrier	java.util.concurrent.CyclicB arrier	Global thread rendezvous synchronization
Thread padding / private	Padded array indexing (stride * 8)	Contiguous padding / @Contended	Cache line isolation (eliminates false sharing)

1.2.Laboratory Environment Configuration

Before attempting any laboratory exercises, students must verify that their execution environment is configured to utilize multi-core hardware resources effectively.

- **Python Environment:** Ensure Python 3.10+ is installed alongside NumPy and Numba. Configure thread parallelism using environment variables: export NUMBA_NUM_THREADS=N and export OMP_NUM_THREADS=N. Verify multi-threading support by executing 'numba.config.NUMBA_NUM_THREADS'.
- **Java Environment:** Ensure OpenJDK 17 or higher is installed. The Java common pool parallelism defaults to Runtime.getRuntime().availableProcessors() - 1. Configure parallelism explicitly via VM argument: -Djava.util.concurrent.ForkJoinPool.common.parallelism=N. For cache padding tests, enable JVM contended fields with: -XX:-RestrictContended.
- **Hardware Specification Reporting:** All lab reports must document the physical CPU model (e.g., AMD Ryzen 9 7950X, Intel Core i7-13700K, Apple M2 Pro), physical core count, logical hardware thread count, and L1/L2/L3 cache sizes.

2.Lab 1: The Fork-Join Model, Team Creation, and Thread Scoping

2.1.Theoretical Foundations & Objectives

In the OpenMP execution model, an application begins execution as a single thread (the master thread). When the master thread encounters a parallel construct ('#pragma omp parallel'), it forks a team of threads. Within the parallel region, each thread is assigned a unique integer rank (Thread ID) ranging from 0 to N-1, where thread 0 is the master. At the conclusion of the parallel block, an implicit barrier synchronizes all threads before the worker threads terminate or return to a dormant pool, leaving only the master thread.

```
jooke@MacBook-Air--jooke lab1_230103077 % python3 lab1.py
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 6569019
[Worker] Logical Rank: 3 of 4 | Native OS TID: 6569021
[Worker] Logical Rank: 1 of 4 | Native OS TID: 6569019
[Worker] Logical Rank: 2 of 4 | Native OS TID: 6569020
--- Joined thread team. Execution returned to serial master ---
Team execution time: 0.002799 seconds
```

```
jooke@MacBook-Air--jooke lab1_230103077 % for i in {1..10}; do
  echo "===== RUN $i ====="
  python3 lab1.py
done | tee task1_1_output.txt
===== RUN 1 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 6570512
[Worker] Logical Rank: 3 of 4 | Native OS TID: 6570514
[Worker] Logical Rank: 1 of 4 | Native OS TID: 6570512
[Worker] Logical Rank: 2 of 4 | Native OS TID: 6570513
--- Joined thread team. Execution returned to serial master ---
Team execution time: 0.002776 seconds

===== RUN 2 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 6570519
[Worker] Logical Rank: 3 of 4 | Native OS TID: 6570521
[Worker] Logical Rank: 1 of 4 | Native OS TID: 6570519
[Worker] Logical Rank: 2 of 4 | Native OS TID: 6570520
--- Joined thread team. Execution returned to serial master ---
Team execution time: 0.002681 seconds

===== RUN 3 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 6570526
[Worker] Logical Rank: 3 of 4 | Native OS TID: 6570527
[Worker] Logical Rank: 1 of 4 | Native OS TID: 6570526
[Worker] Logical Rank: 2 of 4 | Native OS TID: 6570528
--- Joined thread team. Execution returned to serial master ---
Team execution time: 0.002779 seconds

===== RUN 4 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 6570533
[Worker] Logical Rank: 3 of 4 | Native OS TID: 6570534
[Worker] Logical Rank: 1 of 4 | Native OS TID: 6570533
[Worker] Logical Rank: 2 of 4 | Native OS TID: 6570535
--- Joined thread team. Execution returned to serial master ---
Team execution time: 0.002707 seconds

===== RUN 5 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 6570540
[Worker] Logical Rank: 3 of 4 | Native OS TID: 6570540
[Worker] Logical Rank: 1 of 4 | Native OS TID: 6570541
[Worker] Logical Rank: 2 of 4 | Native OS TID: 6570542
--- Joined thread team. Execution returned to serial master ---
Team execution time: 0.002811 seconds
```

Learning Goals: Key Objectives:

- Implement the fork-join parallel lifecycle and verify thread team instantiation.
- Demonstrate non-deterministic execution ordering across operating system thread schedulers.
- Analyze the performance implications of hardware oversubscription (running more software threads than physical cores).

2.2.Directives Tested & Architectural Equivalents

- **OpenMP Directive:** #pragma omp parallel: Instantiates a team of threads.
- **OpenMP Function:** omp_get_thread_num(): Retrieves the calling thread's unique integer rank.
- **OpenMP Function:** omp_get_num_threads(): Retrieves the total number of threads in the team.
- **OpenMP Function:** omp_set_num_threads(N): Sets the thread team size programmatically.

```
jooke@MacBook-Air--jooke lab1_230103077 % python3 benchmark.py
LAB 1 – TASK 1.2
Thread Oversubscription Sweep

P,Execution_Time_Seconds
1,0.000303
2,0.000544
4,0.001097
8,0.001919
16,0.003667
32,0.007261
64,0.015041
```

Threads P	Average time (s)
1	0.000303
2	0.000544
4	0.001097
8	0.001919
16	0.003667
32	0.007261
64	0.015041

2.3.Starter Code Templates

Python Option: Python Starter Implementation (Thread Identification & Fork-Join Lifecycle):

```
// --- Python 3: Fork-Join Team Emulation ---
from concurrent.futures import ThreadPoolExecutor
import threading
import time
import os

def worker_task(thread_id: int, team_size: int):
    # Retrieve OS-level native thread ID
    native_tid = threading.get_native_id()
    role = "Master" if thread_id == 0 else "Worker"
    # Demonstrate execution interleaving
    time.sleep(0.001 * (thread_id % 3))
    print(f"[{role}] Logical Rank: {thread_id} of {team_size} | Native OS TID: {native_tid}")

def run_team(num_threads: int):
    print(f"--- Forking a team of {num_threads} threads ---")
    with ThreadPoolExecutor(max_workers=num_threads) as executor:
        futures = [executor.submit(worker_task, tid, num_threads) for tid in range(num_threads)]
        for f in futures:
            f.result() # Wait for all threads (implicit barrier)
    print("--- Joined thread team. Execution returned to serial master ---\n")

if __name__ == "__main__":
    run_team(4)
```

```
[jooke@MacBook-Air--jooke lab1_230103077 % python3 fork_join.py
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 6581546
[Worker] Logical Rank: 3 of 4 | Native OS TID: 6581547
[Worker] Logical Rank: 1 of 4 | Native OS TID: 6581546
[Worker] Logical Rank: 2 of 4 | Native OS TID: 6581548
--- Joined thread team. Execution returned to serial master ---
```

2.4. Step-by-Step Student Implementation Tasks

- **Task 1.1: Verification of Non-Determinism:** Compile and run the starter template. Execute the program 10 consecutive times without modifying the source code. Save the stdout outputs to a text file.
- **Task 1.2: Thread Oversubscription Sweep:** Modify the thread team size parameter across values P in $\{1, 2, 4, 8, 16, 32, 64\}$. Measure the wall-clock execution time required to instantiate and join the thread team for each value of P .
- **Task 1.3: CPU Core Saturation Analysis:** Introduce an artificial workload inside each thread's execution block (e.g., computing 10,000,000 floating-point square roots). Record CPU utilization across all cores using your OS system monitor (htop, Task Manager, or Activity Monitor).

```
jooke@MacBook-Air--jooke lab1_230103077 % for i in {1..10}; do
    echo "==== RUN $i ====="
    python3 fork_join.py
[done > task1_1_outputs.txt
```

Run	Execution Time (s)	Output Order
1	0.002776	0 → 3 → 1 → 2
2	0.002681	0 → 3 → 1 → 2
3	0.002779	0 → 3 → 1 → 2
4	0.002707	0 → 3 → 1 → 2
5	0.002811	0 → 3 → 1 → 2
6	0.002790	0 → 3 → 1 → 2
7	0.002750	3 → 0 → 1 → 2
8	0.002749	0 → 3 → 1 → 2
9	0.002786	3 → 0 → 1 → 2
10	0.002707	0 → 3 → 1 → 2

2.5. Analytical Questions & Lab Report Prompts

- **Question 1.1 (Scheduling):** Explain why the thread IDs print in non-sequential order across consecutive runs. Which component of the operating system and CPU architecture governs this behavior?

The IDs of the threads can be printed out of sequence because the decision on which thread will execute first is done by the OS scheduler. The threads can be started nearly simultaneously, but there is no guarantee on their execution sequence.

The key element that causes this effect is the **OS scheduler** because it assigns the threads that are ready to run to available CPU cores. There is also influence from the CPU architecture since the threads are executed on physical cores (and on execution contexts when SMT is used).

- **Question 1.2 (Oversubscription):** What performance penalty occurs when P exceeds the number of physical hardware cores? Explain the concepts of context switching, thread state preservation, and cache thrashing.

Oversubscription occurs when there are more software threads than physical CPU cores. In such cases, more than one thread needs to use the same physical CPU core.

One of the main impacts on performance is **context switching**, where the operating system suspends one thread and switches to another thread. This is done by saving the state of the currently running thread, like register values and program counters, and then loading the state of the next thread.

Another potential problem with too many threads is **cache thrashing**. Data from one thread might be overwritten by data from another thread in CPU caches. This results in higher cache misses and memory access costs. Thus, increasing the number of threads doesn't necessarily speed up the execution of a program.

- **Question 1.3 (Barriers):** In OpenMP, what is the role of the implicit barrier at the end of a parallel region? What hazards would arise if worker threads continued executing into subsequent code blocks before reaching the barrier?

An implicit barrier at the end of the parallel OpenMP region ensures that all the threads complete their work before proceeding with the rest of the sequential portion of the code.

This becomes significant in the situation where the subsequent code fragment relies on the results generated by the worker threads. In the absence of a barrier, it would be possible for some of the threads to still be accessing the shared data while another thread begins its access.

Consequently, race conditions and synchronization issues become likely.

- **Question 1.4 (Architectural Mapping):** Contrast a hardware execution thread (Hyper-Threading / SMT) with an operating system kernel thread and a language-level green/virtual thread.

A hardware execution thread is an execution context of the CPU that is provided by the CPU itself. For example, when using hyper-threading or SMT technology, one CPU core can offer several hardware execution contexts, which will use some shared physical resources of the same core.

An OS kernel thread is a thread that is managed and scheduled by the operating system. In other words, the OS scheduler assigns the kernel thread to a physical CPU core or to a hardware execution thread.

A green or virtual thread is a thread that is not scheduled independently but is mainly managed by the runtime of the programming language. There may be many green threads running on fewer OS threads. This kind of thread is helpful for handling numerous concurrent tasks while keeping the memory and scheduling costs lower.

In other words:

Hardware thread = CPU execution context

Kernel thread = OS thread

Green/virtual thread = runtime thread

2.6.Assessment Rubric

Evaluation Criterion	Max Points	Evaluation Standards
Correctness & Execution	25	Code runs without errors, demonstrates proper team creation and ID discovery.
Oversubscription Data	25	Comprehensive timing table spanning P = 1 to 64 with clear scaling trends.
Analytical Question Responses	35	Deep, technically rigorous explanations of OS scheduling, context switches, and barriers.
Code Quality & Documentation	15	Clean formatting, appropriate comments, and reproduction instructions.

3.Lab 2: Numerical Integration (Pi Approximation) & Parallel Reductions

3.1.Theoretical Foundations & Objectives

Numerical integration via the Midpoint Rule approximates the definite integral of a mathematical function as a Riemann sum. The mathematical constant Pi can be approximated by integrating $f(x) = 4 / (1 + x^2)$ from 0 to 1:

A shared accumulator without synchronization can produce a race condition. Critical sections prevent simultaneous updates but introduce synchronization overhead. A parallel reduction uses private accumulators and combines them at the end, which is much more efficient.

Integration Target: Formula: $\text{Integral}[0 \text{ to } 1] \frac{4}{1 + x^2} dx = \pi$

Discretizing this integral into N uniform intervals of width $dx = 1.0 / N$ yields the summation:

Riemann Midpoint Sum: Summation: $\pi \approx dx * \sum_{i=0}^{N-1} \frac{4}{1 + ((i + 0.5) * dx)^2}$

When parallelizing this summation across multiple threads, each thread computes a subset of the terms. If all threads attempt to update a single shared accumulator variable simultaneously without coordination, a critical data race occurs, resulting in incorrect calculations. To resolve this, three distinct synchronization patterns can be utilized: naive unsynchronized updates, mutual exclusion (critical sections), and tree-based parallel reductions.

Learning Goals: Key Objectives:

- Observe and quantify the devastating impact of race conditions on numerical precision.
- Measure the severe performance degradation introduced by lock contention in critical sections.
- Implement hardware-efficient parallel reductions utilizing private thread accumulators.

Run	Calculated Pi	Absolute Error
1	2.42695	0.71464
2	2.29477	0.84682
3	2.38021	0.76138
4	2.38874	0.75285
5	2.27819	0.86340

A non-atomic update such as:

```
total += value
```

is not one indivisible CPU operation. Conceptually, it consists of several steps:

```
LOAD total
ADD value
STORE total
```

Suppose the current value is 100.

Thread A and Thread B can both execute:

```

A: LOAD 100
B: LOAD 100
A: ADD value_A
B: ADD value_B
A: STORE result_A
B: STORE result_B

```

The final store from Thread B overwrites Thread A's result.

Therefore, one update is lost.

This is why the program can produce a value of π that is much lower than the correct value.

Empirically evaluate Amdahl's Law by computing speedup $S(P)$ and parallel efficiency $E(P)$.

3.2.Directives Tested & Architectural Equivalents

- **OpenMP Directive:** `#pragma omp parallel for reduction(+:var)`: Automatically allocates thread-local accumulators and combines them via a binary reduction tree.
- **OpenMP Directive:** `#pragma omp critical`: Enforces strict mutual exclusion, allowing only one thread at a time to execute the enclosed block.
- **OpenMP Directive:** `#pragma omp atomic`: Enforces atomic memory access via specialized CPU instruction primitives (e.g., LOCK ADD or CMPXCHG).

3.3.Starter Code Templates

Python Option: Python Starter Implementation (Numba Reductions & Timing Harness):

For the parallel reduction experiment, the required thread counts are:

$P = 1, 2, 4, 8, 16$

The benchmark should use five trials for every value of P .

Your previously recorded parallel π experiment showed that the calculated value stayed very close to:

$\pi \approx 3.1415$

and the execution time was approximately in the **13.8–14.7 second range** for the recorded fixed-thread experiment.

The important observation is that the reduction produces a correct result while distributing the numerical work between threads.

```
// --- Python 3: Numerical Integration Benchmarking ---
import numpy as np
import time
from numba import njit, prange

N_STEPS = 100_000_000

# 1. Serial Baseline
@njit
def calc_pi_serial(num_steps: int) -> float:
    step = 1.0 / num_steps
    total_sum = 0.0
    for i in range(num_steps):
        x = (i + 0.5) * step
        total_sum += 4.0 / (1.0 + x * x)
    return total_sum * step

# 2. Parallel Reduction (Numba compiles prange reduction into OpenMP-style tree reduction)
@njit(parallel=True)
def calc_pi_reduction(num_steps: int) -> float:
    step = 1.0 / num_steps
    total_sum = 0.0
    for i in prange(num_steps):
        x = (i + 0.5) * step
        total_sum += 4.0 / (1.0 + x * x) # Numba automatically infers parallel reduction
    return total_sum * step

if __name__ == "__main__":
    # Warm-up JIT compilation
    _ = calc_pi_serial(1000)
    _ = calc_pi_reduction(1000)

    # Benchmark Serial
    t0 = time.perf_counter()
    pi_serial = calc_pi_serial(N_STEPS)
    t1 = time.perf_counter()
    time_serial = t1 - t0

    # Benchmark Parallel Reduction
    t2 = time.perf_counter()
    pi_parallel = calc_pi_reduction(N_STEPS)
    t3 = time.perf_counter()

    time_parallel = t3 - t2

    print(f"Serial:   Pi = {pi_serial:.12f} | Time = {time_serial:.4f}s | Error = {abs(pi_serial - np.pi):.2e}")
    print(f"Parallel: Pi = {pi_parallel:.12f} | Time = {time_parallel:.4f}s | Error = {abs(pi_parallel - np.pi):.2e}")
    print(f"Observed Speedup: {time_serial / time_parallel:.2f}x")
```

Java Option: Java Starter Implementation (Race Condition, Synchronized Lock, and Parallel Streams Reduction):

```
// --- Java 17+: Three Variants of Pi Integration ---
import java.util.concurrent.ForkJoinPool;
import java.util.stream.LongStream;

public class PiIntegrationLab2 {
    static final long N = 100_000_000L;
```

```

static final double STEP = 1.0 / N;

// Variant A: Naive Unsynchronized Race Condition
static double runNaiveRace(int threads) throws InterruptedException
{
    double[] sharedSum = new double[1];
    Thread[] pool = new Thread[threads];
    long chunkSize = N / threads;

    for (int t = 0; t < threads; t++)
    {
        final long start = t * chunkSize;
        final long end = (t == threads - 1) ? N : start + chunkSize;
        pool[t] = new Thread(() -> {
            for (long i = start; i < end; i++)
            {
                double x = (i + 0.5) * STEP;
                sharedSum[0] += 4.0 / (1.0 + x * x); // Unprotected shared write
            }
        });
        pool[t].start();
    }
    for (Thread t : pool) t.join();
    return sharedSum[0] * STEP;
}

// Variant B: Synchronized Critical Section (Lock per step)
static double runCriticalSection(int threads) throws InterruptedException
{
    final Object lock = new Object();
    double[] sharedSum = new double[1];
    Thread[] pool = new Thread[threads];
    long chunkSize = N / threads;

    for (int t = 0; t < threads; t++)
    {
        final long start = t * chunkSize;
        final long end = (t == threads - 1) ? N : start + chunkSize;
        pool[t] = new Thread(() -> {
            for (long i = start; i < end; i++)
            {
                double x = (i + 0.5) * STEP;
                double term = 4.0 / (1.0 + x * x);
                synchronized (lock) { // Emulates #pragma omp critical
                    sharedSum[0] += term;
                }
            }
        });
    }
}

```

```

        pool[t].start();
    }
    for (Thread t : pool) t.join();
    return sharedSum[0] * STEP;
}

// Variant C: Parallel Reduction (Emulates #pragma omp parallel for reduction(+:sum))
static double runParallelReduction() {
    double sum = LongStream.range(0, N)
        .parallel()
        .mapToDouble(i -> {
            double x = (i + 0.5) * STEP;
            return 4.0 / (1.0 + x * x);
        })
        .sum();
    return sum * STEP;
}

public static void main(String[] args) throws Exception {
    System.out.println("Starting Numerical Integration Benchmark (N = " + N + ")...");

    // 1. Parallel Reduction
    long t0 = System.nanoTime();
    double piRed = runParallelReduction();
    long t1 = System.nanoTime();
    System.out.printf("Reduction: Pi = %.12f | Time = %d ms | Error = %.2e%n",
        piRed, (t1 - t0) / 1_000_000, Math.abs(piRed - Math.PI));

    // 2. Naive Race Condition (with 4 threads)
    long t2 = System.nanoTime();
    double piRace = runNaiveRace(4);
    long t3 = System.nanoTime();
    System.out.printf("Naive Race: Pi = %.12f | Time = %d ms | Error = %.2e%n",
        piRace, (t3 - t2) / 1_000_000, Math.abs(piRace - Math.PI));
}
}

```

3.4.Step-by-Step Student Implementation Tasks

- **Task 2.1: Race Condition Quantification:** Execute Variant A (Naive Race) with $P = 1, 2, 4, 8$ threads. Tabulate the resulting calculated Pi value and compute the absolute numerical error relative to true Pi. Explain why increasing the thread count increases the error.
- **Task 2.2: Critical Section Overhead:** Execute Variant B (Critical Section) with $N = 1,000,000$ steps. Benchmark execution time against the single-threaded baseline. Calculate the lock contention overhead percentage.
- **Task 2.3: Strong Scaling Benchmark:** Benchmark Variant C (Parallel Reduction) across thread counts P in $\{1, 2, 4, 8, 16\}$. Record the wall-clock execution time $T(P)$ over 5 trials and calculate the average.
- **Task 2.4: Speedup & Efficiency Modeling:** Compute the Speedup $S(P) = T(1) / T(P)$ and Parallel Efficiency $E(P) = S(P) / P$ for all thread counts. Generate a two-axis line graph displaying Speedup vs. Linear Ideal Speedup.

3.5. Analytical Questions & Lab Report Prompts

- **Question 2.1 (Memory Incoherency):** Explain at the CPU assembly level what occurs during a non-atomic read-modify-write operation (e.g., LOAD, ADD, STORE). Why does thread interleaving cause lost updates in Variant A?
- **Question 2.2 (Reduction Trees):** Why is a binary reduction tree $O(\log P)$ significantly superior in scaling compared to a centralized critical section $O(P)$?
- **Question 2.3 (Amdahl's Law):** According to Amdahl's Law, if 5% of a program's execution time is strictly serial (non-parallelizable), what is the theoretical maximum speedup achievable with an infinite number of processors? If your measured speedup flattens earlier, what secondary architectural factors are responsible?
- **Question 2.4 (Hardware Primitives):** How do modern 64-bit multi-core processors implement atomic memory updates at the hardware bus level (e.g., bus snooping, MESI cache locking, LOAD-LINK/STORE-CONDITIONAL)?

3.6. Assessment Rubric

Evaluation Criterion	Max Points	Evaluation Standards
Implementation of All 3 Variants	25	Correct, complete code for naive race, critical section, and parallel reduction.
Benchmarking & Accuracy Data	25	Accurate data tables showing errors, times, speedups, and efficiency across thread counts.
Speedup & Scaling Graphs	20	Properly labeled graphs comparing empirical speedup to theoretical linear speedup.
Theoretical Analysis & Amdahl's Law	30	Rigorous answers to questions on assembly RMW cycles, reduction complexity, and Amdahl limits.

4. Lab 3: Work-Sharing & Loop Scheduling Policies (Mandelbrot Fractal)

4.1. Theoretical Foundations & Objectives

In scientific computing, loop iterations rarely require identical execution times. When computational cost per iteration is uniform, a static distribution (dividing N iterations into equal blocks of size N/P) provides minimal overhead. However, when the workload is highly non-uniform, static scheduling causes load imbalance: threads assigned computationally cheap iterations finish early and sit idle at the barrier, while 'straggler' threads assigned dense iterations bottleneck total execution.

The Mandelbrot Set escape-time algorithm is a classic non-uniform benchmark. For each pixel coordinate $(x0,$

y_0), the recurrence $z_{k+1} = z_k^2 + c$ is evaluated until either $|z| > 2$ (divergence) or a

The Mandelbrot fractal is a good example of a non-uniform workload. Different pixels need different numbers of iterations. Pixels outside the Mandelbrot set usually escape after only a few iterations, while pixels inside the set can require up to 1000 iterations.

Because of this difference, static scheduling can create load imbalance. Some threads may finish their assigned rows early, while other threads continue working on more computationally expensive rows.

In this lab, I studied three OpenMP scheduling policies:

- **Static** — work is divided into fixed chunks before execution.
- **Dynamic** — threads take new chunks from a shared work queue when they finish.
- **Guided** — the chunks start large and become smaller during execution.

The main goal was to compare scheduling policies and study how thread count and chunk size affect execution time.

maximum iteration count (e.g., 1,000) is reached. Pixels outside the set escape after only 2 to 5 iterations, whereas pixels inside the set require the full 1,000 iterations. Because adjacent rows in the complex plane contain vastly differing ratios of interior to exterior points, naive static loop partitioning creates severe load imbalance.

Learning Goals: Key Objectives:

- Implement and contrast OpenMP loop scheduling policies: Static, Dynamic, and Guided.
- Evaluate the performance trade-offs of chunk size granularity C in $\{1, 16, 64, 256\}$.
- Analyze the synchronization overhead of dynamic self-scheduling queues versus thread idling penalty.

4.2.Directives Tested & Architectural Equivalents

- **OpenMP Directive:** `schedule(static, chunk)`: Divides iterations into fixed-size chunks distributed round-robin at compile/start time.
- **OpenMP Directive:** `schedule(dynamic, chunk)`: Puts chunks of iterations into a shared queue; threads dynamically claim a chunk when idle.
- **OpenMP Directive:** `schedule(guided, chunk)`: Starts with large chunk sizes to minimize scheduling overhead, exponentially tapering down to chunk as work completes.

Threads P	C = 1	C = 16	C = 64	C = 256
2	measure d	measure d	measure d	measure d
4	measure d	measure d	measure d	measure d
8	measure d	measure d	measure d	measure d
16	measure d	measure d	measure d	measure d

4.3.Starter Code Templates

I would use a grouped bar chart or a heatmap to compare the execution times.

The horizontal axis represents the chunk size:

1, 16, 64, 256

The vertical axis represents execution time in seconds or milliseconds.

Different bars/lines represent different thread counts:

2, 4, 8, 16

The visualization makes it easier to see how changing the thread count and chunk size affects Mandelbrot rendering performance.

The expected trend is that very small chunks can suffer from scheduling overhead, while very large chunks can suffer from load imbalance.

Python Option: Python Starter Implementation (Static vs. Dynamic Mandelbrot Harness):

```
// --- Python 3: Mandelbrot Static Work-Sharing ---
import numpy as np
import time
from numba import njit, prange

WIDTH, HEIGHT = 1920, 1080
MAX_ITER = 1000

@njit
def compute_pixel(px, py, width, height, max_iter):
    x0 = (px - width / 2.0) * 4.0 / width
    y0 = (py - height / 2.0) * 4.0 / height
    x, y = 0.0, 0.0
    iteration = 0
    while x * x + y * y <= 4.0 and iteration < max_iter:
        xtemp = x * x - y * y + x0
        y = 2.0 * x * y + y0
        x = xtemp
        iteration += 1
    return iteration

# 1. Static Scheduling: Uniform row distribution
@njit(parallel=True)
def render_mandelbrot_static(width, height, max_iter):
    img = np.zeros((height, width), dtype=np.int32)
    for y in prange(height): # prange applies static chunking by default
        for x in range(width):
            img[y, x] = compute_pixel(x, y, width, height, max_iter)
    return img
```

```

if __name__ == "__main__":
    print(f"Rendering Mandelbrot ({WIDTH}x{HEIGHT}, Max Iter: {MAX_ITER})...")
    # Warm-up
    _ = render_mandelbrot_static(100, 100, 100)

    t0 = time.perf_counter()
    image = render_mandelbrot_static(WIDTH, HEIGHT, MAX_ITER)
    t1 = time.perf_counter()
    print(f"Static Schedule Render Completed in: {t1 - t0:.4f} seconds")

```

Java Option: Java Starter Implementation (Dynamic Work-Stealing Loop Scheduler with Configurable Chunking):

```

// --- Java 17+: Dynamic Work-Stealing Loop Scheduler ---
import java.util.concurrent.atomic.AtomicInteger;

public class MandelbrotSchedulerLab3 {
    static final int WIDTH = 1920, HEIGHT = 1080, MAX_ITER = 1000;
    static final int[][] IMAGE = new int[HEIGHT][WIDTH];

    static int computePixel(int px, int py) {
        double x0 = (px - WIDTH / 2.0) * 4.0 / WIDTH;
        double y0 = (py - HEIGHT / 2.0) * 4.0 / HEIGHT;
        double x = 0.0, y = 0.0;
        int iter = 0;
        while (x * x + y * y <= 4.0 and iter < MAX_ITER)
            { double temp = x * x - y * y + x0;
              y = 2.0 * x * y + y0;
              x = temp;
              iter++;
            }
        return iter;
    }

    // Dynamic scheduling with configurable chunk size
    public static long runDynamic(int numThreads, int chunkSize) throws
    InterruptedException {
        AtomicInteger workQueue = new AtomicInteger(0); // Emulates dynamic work dispenser
        Thread[] threads = new Thread[numThreads];
        long startTime = System.currentTimeMillis();

        for (int t = 0; t < numThreads; t++)
            { threads[t] = new Thread(() -> {
                int startRow;
                while ((startRow = workQueue.getAndAdd(chunkSize)) < HEIGHT) {
                    int endRow = Math.min(startRow + chunkSize, HEIGHT);
                    for (int y = startRow; y < endRow; y++)
                        { for (int x = 0; x < WIDTH; x++) {
                            IMAGE[y][x] = computePixel(x, y);
                        }
                    }
                }
            });
            threads[t].start();
        }
        for (Thread t : threads) t.join();
        return System.currentTimeMillis() - startTime;
    }
}

```

```

public static void main(String[] args) throws Exception
{
    int threads = 4;
    int[] chunkSizes = {1, 16, 64, 256};
    System.out.println("Benchmarking Dynamic Loop Scheduling across chunk sizes...");
    for (int chunk : chunkSizes) {
        long duration = runDynamic(threads, chunk);
        System.out.printf("Threads: %d | Chunk Size: %3d | Time: %d ms\n", threads,
            chunk, duration);
    }
}

```

4.4. Step-by-Step Student Implementation Tasks

- **Task 3.1: Scheduler Construction:** Implement both Static and Dynamic loop schedulers in your chosen language.
- **Task 3.2: Parameter Sweep Matrix:** Benchmark execution time across a 4x4 matrix of configurations: Thread Counts P in {2, 4, 8, 16} paired with Chunk Sizes C in {1, 16, 64, 256}. Record 3 runs per cell and calculate the mean.
- **Task 3.3: Data Visualization:** Construct a 2D performance heatmap or grouped bar chart displaying execution time as a function of chunk size and thread count.
- **Task 3.4: Load Imbalance Quantification:** Instrument the code to count the total number of iterations executed by each individual thread. Calculate the load imbalance metric: $\text{Imbalance} = (\text{Max_Thread_Work} - \text{Min_Thread_Work}) / \text{Average_Thread_Work}$.

To measure load imbalance, I counted the total amount of work performed by every thread.

The imbalance formula is:

$$\text{Imbalance} = (\text{Max Thread Work} - \text{Min Thread Work}) / \text{Average Thread Work}$$

A value close to **0** means that the workload is well balanced.

A larger value means that some threads performed significantly more work than others.

Static scheduling can have a larger imbalance because each thread receives a fixed part of the image. Dynamic scheduling can reduce this problem because threads receive new chunks when they become available.

4.5. Analytical Questions & Lab Report Prompts

- **Question 3.1 (Fine-Grained Contention):** Why does dynamic scheduling with chunk size C = 1 exhibit poor performance despite offering theoretically perfect load balancing? Identify the exact hardware resource being contested.
- **Question 3.2 (Spatial Imbalance):** In static scheduling, which thread ranks typically become stragglers when rendering the central section of the Mandelbrot set? Explain using the geometry of the complex plane.
- **Question 3.3 (Guided Policy):** Describe the mathematical mechanism of OpenMP's

schedule(guided, chunk). How does it combine the low overhead of static scheduling with the dynamic adaptation of small chunking?

- **Question 3.4 (Engineering Decision Framework):** Formulate a rule-of-thumb heuristic for software engineers: under what workload characteristics should one select Static vs. Dynamic vs. Guided loop scheduling?

4.6.Assessment Rubric

Evaluation Criterion	Max Points	Evaluation Standards
Scheduling Harness Implementation	25	Fully functional static and dynamic schedulers with configurable chunk
Comprehensive Benchmark Matrix	25	size parameters. Complete 4x4 parameter sweep with accurate timings and load imbalance computations.
Analytical Interpretation	35	Thorough discussion of atomic contention, memory bus bottlenecks, and guided scheduling.
Visual Plots & Code Quality	15	Clear heatmaps/charts with proper axis labeling, clean code structure.

5.Lab 4: Memory Hierarchy, Cache Coherency, and False Sharing

5.1.Theoretical Foundations & Objectives

In symmetric multiprocessing (SMP) systems, each CPU core possesses private L1 and L2 caches, while sharing an L3 cache and system RAM. Data is transferred between cache levels in fixed-size blocks called cache lines (standardly 64 bytes). Cache coherency protocols (such as MESI: Modified, Exclusive, Shared, Invalid) ensure that all cores maintain a consistent view of memory.

False sharing is a critical performance hazard that arises when multiple threads running on distinct physical cores modify independent variables that happen to reside on the same 64-byte cache line. Although the program logic is perfectly thread-safe and free of data races, the hardware cache coherency mechanism detects writes to the shared line. As a result, each core repeatedly invalidates the L1 cache lines of all other cores, forcing expensive bus snooping and memory flushes ('cache-line bouncing').

Learning Goals: Key Objectives:

- Synthetically induce and measure the performance degradation caused by false sharing.
- Eliminate false sharing using cache line padding (stride alignment) and thread-local registers.
- Analyze the MESI cache protocol state transitions responsible for hardware-level bus contention.

5.2.Directives Tested & Architectural Equivalents

- **OpenMP Principle:** Data Layout & Cache Alignment: Isolating independent thread variables onto separate 64-byte cache lines.
- **Software Pattern:** Thread-Private Accumulators: Storing temporary counter values in CPU core registers before writing final results.
- **JVM Mechanism:** Java @Contended / Padding: Enforcing 128-byte object padding to eliminate false sharing on modern JVMs.

5.3.Starter Code Templates

Python Option: Python Starter Implementation (Unpadded vs. Padded Cache Stride Benchmark):

```
// --- Python 3: False Sharing vs. Padded Layout Benchmark ---
import numpy as np
import time
from numba import njit, prange

ITERATIONS = 100_000_000

# 1. False Sharing: Contiguous 64-bit integers share the same 64-byte cache line (8 bytes * 8 = 64 bytes)
@njit(parallel=True)
def false_sharing_test(num_threads: int, iters: int):
    counters = np.zeros(num_threads, dtype=np.int64)
    for tid in prange(num_threads):
        for _ in range(iters):
            counters[tid] += 1
    return counters

# 2. Cache Line Padded: Spacing indices by 8 elements (8 elements * 8 bytes = 64 bytes)
@njit(parallel=True)
def padded_sharing_test(num_threads: int, iters: int):
    stride = 8 # Exactly one 64-byte cache line per thread counter
    counters = np.zeros(num_threads * stride, dtype=np.int64)
    for tid in prange(num_threads):
        idx = tid * stride
        for _ in range(iters):
            counters[idx] += 1
    return counters

if __name__ == "__main__":
    P = 4
    # Warm-up
    _ = false_sharing_test(P, 1000)
    _ = padded_sharing_test(P, 1000)

    print(f"Running False Sharing Benchmark ({P} threads, {ITERATIONS:,} increments/
thread)...")

    t0 = time.perf_counter()
    false_sharing_test(P, ITERATIONS)
    t1 = time.perf_counter()
    time_false = t1 - t0

    t2 = time.perf_counter()
    padded_sharing_test(P, ITERATIONS)
    t3 = time.perf_counter()
    time_padded = t3 - t2

    print(f"Unpadded (False Sharing): {time_false:.4f} seconds")
    print(f"Padded (Cache Isolated): {time_padded:.4f} seconds")
    print(f"Speedup from Cache Isolation: {time_false / time_padded:.2f}x")
```

Java Option: Java Starter Implementation (Contiguous Array vs. Contended Object / Manual Padding):

```
// --- Java 17+: False Sharing vs. Manual Cache Padding ---
public class FalseSharingLab4 {
    static final long ITERATIONS = 100_000_000L;
    static final int NUM_THREADS = 4;

    // Unpadded: Adjacent 64-bit longs reside in the same 64-byte cache line
```

```

static class UnpaddedCounters {
    public volatile long[] values = new long[NUM_THREADS];
}

// Padded: 64 bytes between counters (7 dummy longs * 8 bytes + 8 bytes counter = 64
bytes)
static class PaddedCounters {
    public volatile long[] values = new long[NUM_THREADS * 8];
}

public static void main(String[] args) throws Exception {
    // Benchmark Unpadded
    UnpaddedCounters unpadded = new UnpaddedCounters();
    Thread[] threadsA = new Thread[NUM_THREADS];
    long t0 = System.currentTimeMillis();
    for (int t = 0; t < NUM_THREADS; t++) {
        final int tid = t;
        threadsA[t] = new Thread(() -> {
            for (long i = 0; i < ITERATIONS; i++)
                { unpadded.values[tid]++;
            }
        });
        threadsA[t].start();
    }
    for (Thread t : threadsA) t.join();
    long unpaddedTime = System.currentTimeMillis() - t0;

    // Benchmark Padded
    PaddedCounters padded = new PaddedCounters();
    Thread[] threadsB = new Thread[NUM_THREADS];
    long t1 = System.currentTimeMillis();
    for (int t = 0; t < NUM_THREADS; t++) {
        final int idx = t * 8; // 64-byte spacing
        threadsB[t] = new Thread(() -> {
            for (long i = 0; i < ITERATIONS; i++) { padded.values[idx]
                ++;
            }
        });
        threadsB[t].start();
    }
    for (Thread t : threadsB) t.join();
    long paddedTime = System.currentTimeMillis() - t1;

    System.out.printf("Unpadded (False Sharing): %d ms\n", unpaddedTime);
    System.out.printf("Padded (Isolated Line):      %d ms\n", paddedTime);
    System.out.printf("Speedup Ratio:                %.2fx\n", (double) unpaddedTime /
paddedTime);
}

```

5.4.Step-by-Step Student Implementation Tasks

- **Task 4.1: Unpadded Scaling Profile:** Execute the unpadded false sharing benchmark across thread counts P in $\{1, 2, 4, 8, 16\}$. Plot execution time vs. thread count. Observe whether execution time increases as more threads are added.
- **Task 4.2: Padded Scaling Profile:** Execute the padded benchmark across the exact same thread counts. Add the padded curves to your scaling graph.

- **Task 4.3: Thread-Local Register Optimization:** Implement a third variant: Thread-Local Accumulator. In this variant, each thread increments a local scalar register and writes to the shared array exactly once upon loop termination.
- **Task 4.4: Hardware Performance Profiler Analysis:** If running Linux, profile hardware performance counters using 'perf stat -e L1-dcache-load-misses,L1-dcache-store-misses ./executable'. Document the cache miss counts.

5.5. Analytical Questions & Lab Report Prompts

- **Question 4.1 (MESI State Transitions):** Diagram a 64-byte cache line containing four 64-bit integer values. Trace the sequence of MESI state transitions (Modified, Exclusive, Shared, Invalid) when Core 0 and Core 1 alternate write operations.
- **Question 4.2 (Bus Bouncing):** Why does adding more CPU cores in the unpadded test cause execution time to become slower than a single thread? What is the physical bottleneck in the processor architecture?
- **Question 4.3 (True vs. False Sharing):** Explain the difference between 'True Sharing' (two threads accessing the identical variable) and 'False Sharing' (two threads accessing different variables on the same cache block).
- **Question 4.4 (Adjacent Line Prefetching):** Research the JVM '@jdk.internal.vm.annotation.Contended' annotation. Why does the JVM enforce 128 bytes of padding instead of 64 bytes by default on x86-64 architectures?

5.6. Assessment Rubric

Evaluation Criterion	Max Points	Evaluation Standards
Benchmark Implementation	25	Flawless execution of unpadded, padded, and local-accumulator variants.
Scaling Plots & Hardware Metrics	25	Accurate graphs demonstrating the slowdown curve of false sharing and recovery with padding.
MESI Protocol Analysis	35	Rigorous step-by-step diagram and technical explanation of cache coherency invalidation states.
Code Quality & Reproducibility	15	Well-documented source code, clear compilation instructions, and hardware disclosure.

6.Lab 5: Recursive Task-Based Parallelism (Parallel Merge Sort)

6.1.Theoretical Foundations & Objectives

Traditional OpenMP loop work-sharing constructs are restricted to canonical loops with predetermined iteration counts. For irregular computations—such as recursive divide-and-conquer algorithms, pointer-chasing graph traversals, and dynamic trees—OpenMP 3.0 introduced task-based parallelism ('#pragma omp task'). Under this paradigm, units of independent computation are instantiated as discrete tasks and placed into a dynamic work-stealing task pool.

Parallel Merge Sort divides an array of size N into two halves, recursively sorts each half asynchronously as independent tasks, and merges the sorted sub-arrays. However, generating tasks down to single-element base cases generates millions of tiny tasks, causing scheduling and allocation overhead to swamp computational throughput. To achieve optimal performance, an adaptive sequential cutoff threshold K must be utilized, below which execution transitions to high-speed sequential sorting.

Learning Goals: Key Objectives:

- Implement divide-and-conquer task parallelism with fork-join synchronization barriers.
- Experimentally determine the optimal sequential cutoff threshold K .
- Perform Work-Span (Critical Path) analysis to determine the theoretical upper-bound on parallelism.

6.2.Directives Tested & Architectural Equivalents

- **OpenMP Directive:** #pragma omp task: Packages a computational block into an independent task for deferred/asynchronous execution.
- **OpenMP Directive:** #pragma omp taskwait: Suspends the calling task until all child tasks spawned by the current task have completed.
- **Java Mechanism:** Java ForkJoinPool / RecursiveAction: Industry-standard work-stealing scheduler implementing the Cilk/OpenMP task model.

6.3.Starter Code Templates

Java Option: Java Starter Implementation (ForkJoinTask Divide-and-Conquer Merge Sort):

```
// --- Java 17+: Parallel Merge Sort with Work-Stealing Tasks ---
import java.util.Arrays;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

public class ParallelMergeSortLab5 extends RecursiveAction
{ private final int[] array;
  private final int left, right;
  private final int cutoff;

  public ParallelMergeSortLab5(int[] array, int left, int right, int cutoff)
  { this.array = array;
    this.left = left;
```

```

        this.right = right;
        this.cutoff = cutoff;
    }

    @Override
    protected void compute() {
        // Sequential cutoff threshold (OpenMP task granularity control)
        if (right - left <= cutoff) {
            Arrays.sort(array, left, right + 1);
            return;
        }

        int mid = left + (right - left) / 2;

        // Emulates #pragma omp task
        ParallelMergeSortLab5 leftTask = new ParallelMergeSortLab5(array, left, mid,
cutoff);
        // Emulates #pragma omp task
        ParallelMergeSortLab5 rightTask = new ParallelMergeSortLab5(array, mid + 1, right,
cutoff);

        // Emulates #pragma omp taskwait (spawns both and waits for child completion)
        invokeAll(leftTask, rightTask);

        merge(mid);
    }

    private void merge(int mid) {
        int[] temp = Arrays.copyOfRange(array, left, right + 1);
        int i = 0, j = mid - left + 1, k = left;
        int midOffset = mid - left;
        int endOffset = right - left;

        while (i <= midOffset && j <= endOffset) {
            if (temp[i] <= temp[j]) array[k++] = temp[i++];
            else array[k++] = temp[j++];
        }
        while (i <= midOffset) array[k++] = temp[i++];
        while (j <= endOffset) array[k++] = temp[j++];
    }

    public static void main(String[] args) {
        int size = 5_000_000;
        int[] data = new int[size];
        Arrays.setAll(data, i -> (int) (Math.random() * 10_000_000));

        ForkJoinPool pool = new ForkJoinPool();
        int[] testCutoffs = {10, 100, 1_000, 10_000, 100_000};

        System.out.println("Benchmarking Parallel Merge Sort with Cutoff Thresholds (N = "
+ size + ")...");
        for (int cut : testCutoffs) {
            int[] copy = Arrays.copyOf(data, data.length);
            long start = System.currentTimeMillis();
            pool.invoke(new ParallelMergeSortLab5(copy, 0, copy.length - 1, cut));
            long elapsed = System.currentTimeMillis() - start;
            System.out.printf("Cutoff: %6d | Execution Time: %d ms%n", cut, elapsed);
        }
    }
}

```

Python Option: Python Starter Implementation (Process-Pool Recursive Merge Sort with Cutoff):

```
// --- Python 3: Task-Based Recursive Merge Sort ---
import numpy as np
import time
from concurrent.futures import ProcessPoolExecutor

def sequential_merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = sequential_merge_sort(arr[:mid])
    right = sequential_merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def parallel_merge_sort_entry(arr, cutoff=50000):
    if len(arr) <= cutoff:
        return sequential_merge_sort(arr)
    mid = len(arr) // 2

    # Process-based asynchronous tasks
    with ProcessPoolExecutor(max_workers=2) as executor:
        f_left = executor.submit(parallel_merge_sort_entry, arr[:mid], cutoff)
        f_right = executor.submit(parallel_merge_sort_entry, arr[mid:], cutoff)
        left = f_left.result()
        right = f_right.result()
    return merge(left, right)

if __name__ == "__main__":
    N = 1_000_000
    data = np.random.randint(0, 10_000_000, size=N).tolist()

    t0 = time.perf_counter()
    sorted_arr = parallel_merge_sort_entry(data, cutoff=50_000)
    t1 = time.perf_counter()
    print(f"Parallel Sort Completed (N={N}): {t1 - t0:.4f}s")
```

6.4.Step-by-Step Student Implementation Tasks

- **Task 5.1: Algorithm Verification:** Implement the parallel recursive merge sort. Verify that the output array is properly sorted in ascending order using an assertion check.

- **Task 5.2: Cutoff Threshold Optimization:** Conduct a parametric sweep of the cutoff threshold K in $\{1, 10, 100, 1,000, 10,000, 50,000, 100,000\}$ on an array of 5,000,000 integers. Plot execution time as a function of $\log(K)$.
- **Task 5.3: Work-Span Model Calculation:** Compute the Work (T_1) and Span (T_∞) for recursive merge sort. Work is the total time on 1 core; Span is the longest directed acyclic path of tasks. Calculate the theoretical parallelism $P_{\text{theoretical}} = T_1 / T_\infty$.

6.5. Analytical Questions & Lab Report Prompts

- **Question 5.1 (Granularity Breakdown):** What catastrophic behavior occurs when cutoff $K = 1$? Explain the memory overhead of task descriptors, stack frames, and queue allocation.
- **Question 5.2 (Work-Stealing Mechanics):** How does a work-stealing scheduler operate? Distinguish between a thread pushing/popping from its own local double-ended queue (deque) vs. stealing from the tail of another thread's deque.
- **Question 5.3 (Span Bottlenecks):** In standard recursive merge sort, the merge step is sequential ($O(N)$). How does this sequential merge phase constrain the critical path (Span)? How could the merge step itself be parallelized?
- **Question 5.4 (Loop vs. Tasking Paradigm):** Compare OpenMP loop work-sharing (`#pragma omp for`) with dynamic tasking (`#pragma omp task`). In what computational problem domains is tasking fundamentally superior?

6.6. Assessment Rubric

Evaluation Criterion	Max Points	Evaluation Standards
Tasking Implementation & Correctness	25	Accurate recursive task splitting, barrier synchronization, and output verification.
Threshold Sweep & Curve	25	Clear graphical representation showing optimal cutoff trough and explaining extremes.
Work-Span Theoretical Analysis	35	Mathematically correct Work T_1 , Span T_∞ , and theoretical parallelism derivations.
Code Organization & Readability	15	Modular, cleanly documented code adhering to high-performance guidelines.

7. Lab 6 (Bonus Challenge): Pipeline Parallelism & Wavefront

Processing

7.1. Theoretical Foundations & Objectives

While data parallelism partitions arrays among threads executing the same operation, functional or pipeline parallelism partitions distinct operational stages among threads operating on continuous data streams. In OpenMP, this is achieved via '#pragma omp sections' or concurrent producer-consumer queues. Each stage consumes output from the preceding stage, processes it, and queues it for the subsequent stage, allowing all pipeline stages to execute concurrently in steady-state.

7.2. Bonus Problem Specification

Challenge Problem: Design and implement a 3-stage stream processing pipeline:

- **Stage 1 (Producer/Reader):** Ingests raw data packets (e.g., synthetic matrix chunks or image frames) from memory.
- **Stage 2 (Compute/Filter):** Applies a computationally intensive filter (e.g., 2D Gaussian blur or Sobel edge detection convolution kernel).
- **Stage 3 (Consumer/Writer):** Calculates statistical metrics (min, max, mean, checksum) and writes results to an output buffer.

Threads must communicate across stages using bounded thread-safe blocking queues (or OpenMP circular ring buffers with mutexes). Measure pipeline latency, throughput (packets/sec), and queue occupancy across varying filter workloads.

7.3. Extra Credit Grading Criteria

- +10 Bonus Points: Fully working 3-stage pipeline with thread-safe synchronized queues and graceful shutdown handling.
- +10 Bonus Points: In-depth report evaluating throughput bottleneck analysis, identifying whether the pipeline is compute-bound (Stage 2) or I/O-bound (Stage 1/3).

8. Laboratory Report Guidelines & Submission Template

8.1. Deliverables Checklist

Every student must commit the following to Git:

- **1. Laboratory Report:** A completed formal lab report compiled in PDF or DOCX format adhering to the structure outlined below.
- **2. Source Code Repository:** All commented source code files (.py, .java, .c), organized into sub-folders for each lab (e.g., /lab1, /lab2, etc.). Must include a README with compile/execution commands.
- **3. Raw Benchmark Data:** Raw performance timing data saved in CSV format, enabling reproduction of all submitted plots and tables.

8.2.Required Report Structure

Your submitted report must contain the following standard sections:

- **Section I: System & Hardware Specifications:** Listing of CPU model, physical cores, logical hardware threads, L1/L2/L3 cache sizes, OS version, and runtime/compiler versions.
- **Section II: Experimental Methodology:** Explanation of execution parameters, timer mechanisms used (e.g., `time.perf_counter()`, `System.nanoTime()`), warm-up iterations, and number of experimental trials.
- **Section III: Empirical Results & Visualizations:** Neatly formatted tables and high-resolution plots for all speedup, efficiency, and sweep benchmarks.
- **Section IV: Analytical & Discussion Responses:** Comprehensive, paragraph-form answers to all numbered analytical questions in Labs 1 through 5.
- **Section V: Conclusions & Insights:** Summary of key takeaways regarding shared-memory concurrency, memory hierarchy bottlenecks, and parallel scheduling.

8.3.Final Grading Summary

Laboratory Module	Maximum Score	Core Focus Area
Lab 1: Fork-Join Model & Thread Teams	15 Points	Thread scoping, non-determinism, oversubscription
Lab 2: Numerical Integration & Reductions	25 Points	Race conditions, critical sections, reduction scaling
Lab 3: Work-Sharing & Loop Scheduling	20 Points	Static vs. dynamic scheduling, chunk size tuning
Lab 4: Cache Architecture & False Sharing	20 Points	MESI protocol, cache line padding, bus invalidation
Lab 5: Task-Based Parallel Merge Sort	20 Points	Dynamic tasking, cutoff optimization, Work-Span
Lab 6 (Bonus Challenge): Pipeline Parallelism	+20 Points Extra	Multi-stage stream queues, throughput profiling
Total Possible Score	100 (+20) Points	Comprehensive Shared-Memory Concurrency Mastery